

ACTIVITY GUIDE

Table of Contents

Chapter 1: Introduction to Machine Learning	3
Overview	4
Chapter 2: Fundamentals of Machine Learning	5
Overview	6
Chapter 3: Machine Learning Techniques: Classification, Regression and Clustering.....	7
Overview	8
Practical Lesson 3.1: Predicting Product Purchase with K-Nearest Neighbors (KNN) Classifier.....	9
Practical Lesson 3.2: Email Spam Classification using Naïve Bayes	14
Practical Lesson 3.3: Decision Tree Algorithm for Predicting Customer Purchases	25
Practical Lesson 3.4: Simple Linear Regression for Predicting Salaries Based on Experience.....	31
Practical Lesson 3.5: Logistic Regression for Classification based on Age and Estimated Salary	37
Practical Lesson 3.6: K-Means Clustering using the Elbow Method	45
Chapter 4: Natural Language Processing.....	50
Practical Lesson 4: Text Preprocessing, Vectorization, and Sentiment Analysis	51

Chapter 1: Introduction to Machine Learning

Practical Lesson 1: Introduction to Machine Learning

Overview

There is no practical for this lesson

Chapter 2: Fundamentals of Machine Learning

Practical Lesson 2: Fundamentals of Machine Learning

Overview

There is no practical for this lesson

Chapter 3: Machine Learning Techniques: Classification, Regression and Clustering

Practical Lesson 3: Machine Learning Techniques: Classification, Regression and Clustering

[Overview](#)

In this lesson, you will understand the practical implementation of the Machine Learning Techniques

Practical Lesson 3.1: Predicting Product Purchase with K-Nearest Neighbors (KNN) Classifier

Overview

We will be using the Social network ad dataset. The dataset contains the details of users in a social networking site to find whether a user buys a product by clicking the ad on the site based on their salary, age, and gender.

Prerequisite

- Python3 should be configured.
- Anaconda Installer should be configured
- Good internet connection

Tasks

Step 1: Import Libraries

Let's start by importing essential libraries:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import sklearn
```

Step 2: Importing a Dataset

Importing of the dataset and slicing it into independent and dependent variables:

```
dataset = pd.read_csv('C:/Users/Mausomi Paul/datasets/Social_Network_Ads.csv')
X = dataset.iloc[:, [1, 2, 3]].values
y = dataset.iloc[:, -1].values
```

- X: This extracts the features from columns with indices 1, 2, and 3:
 - Column 1: Gender (e.g., 'Male', 'Female')
 - Column 2: Age (e.g., 19, 35, 26, ...)
 - Column 3: EstimatedSalary (e.g., 19000, 20000, 43000, ...)
- y: This extracts the target variable from the last column (Purchased), which indicates whether the user bought the product (1) or not (0)

To print the length of the feature matrix (X), you should call the `len()` function

```
print(len(X)) #Print the number of rows
```

400

To print the first few rows of the dataset, you have to use the head() method.

```
print(dataset. head()) #Print the first few rows of the dataframe
```

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

Printing the first few rows of the features(X) and the target variable(y)

```
print(X[:5])

print("\nFirst few rows of target (y):")
print(y[:5])
```

```
[['Male' 19 19000]
 ['Male' 35 20000]
 ['Female' 26 43000]
 ['Female' 27 57000]
 ['Male' 19 76000]]
```

```
First few rows of target (y):
[0 0 0 0 0]
```

Step 3: Encode LabelEncode

Since our dataset contains character variables we have to encode it using LabelEncoder:

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
X[:,0] = le.fit_transform(X[:,0])
```

- **LabelEncoder** is used to convert categorical data into numerical labels.
- Then we initialize a LabelEncoder object named le.
- `X[:, 0]` selects all rows of the first column of the feature matrix X, which likely represents categorical data (e.g., 'Male', 'Female').
- `le.fit_transform()` fits the encoder to the data in the selected column and transforms the data into numerical labels.
- The transformed numerical labels are then assigned back to the corresponding column in the feature matrix X.

Step 4: Train Test and Split on Dataset

We are performing a train test split on the dataset. We are providing the test size as 0.20, that means our training sample contains 320 training set and test sample contains 80 test set.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.20, random_state = 0)
```

- **X**: The feature matrix containing the independent variables.
- **y**: The target variable containing the dependent variable.
- **test_size = 0.20**: This parameter specifies the proportion of the dataset to include in the test split. Here, 20% of the dataset will be used for testing.
- **random_state = 0**: This parameter sets the random seed for reproducibility. It ensures that the data is split in the same way every time the code is run, which is important for reproducible research.

Outputs:

- **X_train**: This variable contains the features for the training set.
- **X_test**: This variable contains the features for the testing set.
- **y_train**: This variable contains the target values for the training set.
- **y_test**: This variable contains the target values for the testing set.

Next, we are doing feature scaling to the training and test set of independent variables for reducing the size to smaller values:

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

- The **StandardScaler** is a tool used for scaling features, which means transforming data so that it fits within a specific scale.
- we create an instance of the **StandardScaler** class and store it in the variable **sc**. This object will be used to scale our data.
- **sc.fit_transform(X_train)**:
- **sc.fit()** computes the mean and standard deviation for each feature in the training set **X_train**.
- **sc.transform()** then applies these computed mean and standard deviation to transform (scale) the training set **X_train**. This ensures that all features have a mean of 0 and a standard deviation of 1.

- Then we only apply the transformation (scaling) to the test set **X_test**. We use **sc.transform()** instead of **sc.fit_transform()** because we want to use the mean and standard deviation computed from the training set to scale the test set. This ensures consistency in scaling between the training and test sets.

Step 5: Training KNN Model

Now we have to create and train the K Nearest Neighbor model with the training set:

```
from sklearn.neighbors import KNeighborsClassifier
classifier = KNeighborsClassifier(n_neighbors=5, metric='euclidean')

classifier.fit(X_train, y_train)
```

```
▼ KNeighborsClassifier
KNeighborsClassifier(metric='euclidean')
```

Imported the KNeighborsClassifier is a type of supervised learning algorithm used for classification tasks.

- Then we created an instance of the **KNeighborsClassifier** class and assigns it to the variable **classifier**.
- **n_neighbors=5**: This parameter specifies the number of neighbors to consider when making predictions. Here, we're using 5 neighbors.
- **metric='euclidean'**: This parameter specifies the distance metric used to measure the distance between points. In this case, we're using the euclidean distance metric.
- Then we trains (fits) the KNN classifier using the training set (X_train, y_train).
- The fit() method takes the training data as input and learns from it to make predictions.

To predicts the target variable for the test set using the trained K Nearest Neighbors (KNN) classifier

```
y_pred = classifier.predict(X_test)
```

```
print(y_pred)
```

```
[0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0 1 0 0 1 0 1 0 1 0 0 0 0 0 0 1 0 0 0 0
 0 0 1 0 0 0 0 1 0 0 1 0 1 1 0 0 1 1 1 0 0 1 0 0 1 0 1 0 1 0 0 0 0 1 0 0 1
 0 0 0 0 1 1]
```

- **classifier**: This is the trained K Nearest Neighbors (KNN) classifier that was previously created and trained on the training data.
- **.predict()**: This method is used to make predictions on new data. It takes the test features (**X_test**) as input and returns the predicted target variable.

- **X_test**: This is the test set containing the independent variables (features) for which predictions are to be made.
- `print(y_pred)` will output the predicted target variable values stored in the `y_pred` variable. Each value in `y_pred` corresponds to the predicted target variable value for a corresponding row in the test set `X_test`. For example, if **y_pred** contains **[0, 1, 1, 0, 1, ...]**, it means that the model predicted the first test sample to belong to class 0, the second test sample to belong to class 1, the third test sample to belong to class 1, and so on.

Step 6: Evaluating the model

We can evaluate our model using the confusion matrix and accuracy score by comparing the predicted and actual test values:

```
from sklearn.metrics import confusion_matrix, accuracy_score
cm = confusion_matrix(y_test, y_pred)
ac = accuracy_score(y_test, y_pred)
```

```
print(cm)
print(ac)
```

```
[[55  3]
 [ 1 21]]
0.95
```

- **confusion_matrix(y_test, y_pred)**: This function computes the confusion matrix to evaluate the accuracy of a classification. It takes the true target values (**y_test**) and the predicted target values (**y_pred**) as input and returns a confusion matrix.
- **accuracy_score(y_test, y_pred)**: This function calculates the accuracy of the classification model by comparing the true target values (`y_test`) with the predicted target values (`y_pred`). It returns the accuracy score, which is the fraction of correctly predicted samples.

The confusion matrix has four values:

- 55: True negatives (TN) - the number of correctly predicted negative instances (actual negative instances correctly predicted as negative).
- 3: False positives (FP) - the number of incorrectly predicted positive instances (actual negative instances incorrectly predicted as positive).
- 1: False negatives (FN) - the number of incorrectly predicted negative instances (actual positive instances incorrectly predicted as negative).
- 21: True positives (TP) - the number of correctly predicted positive instances (actual positive instances correctly predicted as positive).
- The accuracy score is 0.95, indicating that the model correctly predicted 95% of the samples in the test set.

Practical Lesson 3.2: Email Spam Classification using Naïve Bayes

Overview

In this practical, we will be using a dataset that contains the subject of the mail and also specifies whether the mail is spam or not based on 1 and 0 mentioned.

Prerequisite

- Python3 should be configured.
- Anaconda Installer should be configured
- Good internet connection

Tasks

Step 1: Importing Libraries

```
#Importing the Libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report, confusion_matrix
```

Each of the library imported is used for the following tasks:

1. **Pandas:** Data manipulation and analysis library for working with labeled and relational data.
2. **NumPy:** Fundamental package for numerical computing with support for multidimensional arrays and mathematical functions.
3. **Seaborn:** Statistical data visualization library providing a high-level interface for creating attractive graphics.
4. **Matplotlib:** Comprehensive plotting library for creating static, interactive, and animated visualizations in Python.
5. **scikit-learn:** Machine learning library providing tools for classification, regression, clustering, dimensionality reduction, and model evaluation

step 2: Import the Dataset

```
#Importing the Dataset
spam_df = pd.read_csv('emails.csv')
```

- It reads the data from a CSV file named 'emails.csv' into a Pandas DataFrame named spam_df.
- pd.read_csv(): This function is provided by Pandas to read data from CSV files and create a DataFrame.

To see the first and last rows of the dataset use the head and the tail method.

```
print(spam_df.head(10))
print(spam_df.tail(10))
```

	text	spam
0	Subject: naturally irresistible your corporate...	1
1	Subject: the stock trading gunslinger fanny i...	1
2	Subject: unbelievable new homes made easy im ...	1
3	Subject: 4 color printing special request add...	1
4	Subject: do not have money , get software cds ...	1
5	Subject: great nnews hello , welcome to medzo...	1
6	Subject: here ' s a hot play in motion homela...	1
7	Subject: save your money buy getting this thin...	1
8	Subject: undeliverable : home based business f...	1
9	Subject: save your money buy getting this thin...	1
	text	spam
5718	Subject: altos na gas model kim , i know you ...	0
5719	Subject: power market research i came across ...	0
5720	Subject: re : visit to houston fyi - - - - -...	0
5721	Subject: ees risk management presentations for...	0
5722	Subject: re : vacation vince : i just found ...	0
5723	Subject: re : research and development charges...	0
5724	Subject: re : receipts from visit jim , than...	0
5725	Subject: re : enron case study update wow ! a...	0
5726	Subject: re : interest david , please , call...	0
5727	Subject: news : aurora 5 . 2 update aurora ve...	0

To get detailed information about the dataset you can use the describe and info method.

```
print(spam_df.describe())
print(spam_df.info())
```

```

      spam
count  5728.000000
mean    0.238827
std     0.426404
min     0.000000
25%     0.000000
50%     0.000000
75%     0.000000
max     1.000000
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5728 entries, 0 to 5727
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0   text    5728 non-null    object
 1   spam    5728 non-null    int64
dtypes: int64(1), object(1)
memory usage: 89.6+ KB
None

```

- **spam_df.describe():** generate descriptive statistics of the numerical columns in your DataFrame **spam_df**. It includes count, mean, standard deviation, minimum, maximum, and various percentiles of numeric data.

The spam column contains only 0 and 1.

Since the mean is 0.238827, it means about 23.88% emails are spam.

Because 25%, 50%, and 75% values are all 0, most emails are ham.

So the dataset is imbalanced: ham emails are more than spam emails.

- **spam_df.info()** display a concise summary of the DataFrame **spam_df**, including the number of non-null values and data types for each column. It helps understand the structure of your dataset, including the presence of missing values and the data types of each feature.

The info() function provides a structural summary of the dataset. It tells us that the dataset contains 5728 email records, 2 columns (text and spam), no missing values, the email content is stored as text (object type), the target label is stored as integers (int64), and the dataset occupies about 89.6 KB of memory.

Step 3: Calculate the percentage of spam and ham (non-spam) emails in your dataset:

```
#Seperates spam and ham emails
ham = spam_df[spam_df['spam'] == 0]
spam = spam_df[spam_df['spam'] == 1]
```

```
#Percentage of Spam and ham emails
print('Spam Percentage =', (len(spam) / len(spam_df)) * 100, '%')
print('Ham Percentage =', (len(ham) / len(spam_df)) * 100, '%')
```

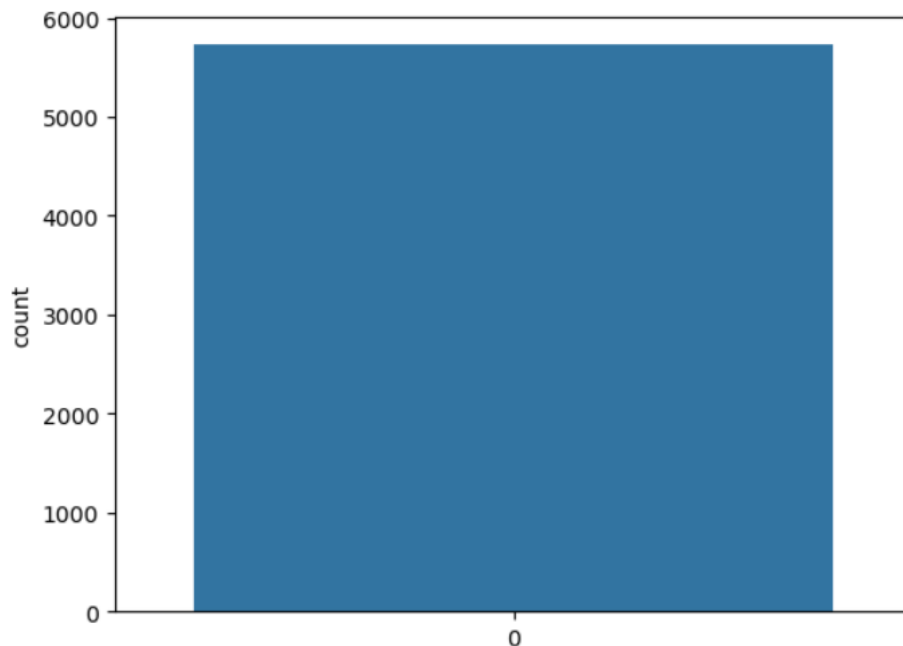
```
Spam Percentage = 23.88268156424581 %
```

```
Ham Percentage = 76.11731843575419 %
```


- We have created a DataFrame ham containing only the rows where the 'spam' column equals 0, indicating non-spam emails.
- Then created a DataFrame spam containing only the rows where the 'spam' column equals 1, indicating spam emails.
- Then we calculated the percentage of spam and non-spam

To visualize the plot you can use the seaborn library

```
#Visualize Spam vs ham
sns.countplot(spam_df['spam'], label='Spam vs Ham')
plt.show()
```



This chart is looking at your email data and separating the emails into two categories: spam and ham.

- On the left side (usually labeled 0), you have the "ham" emails. These are the normal emails you want in your inbox.
- On the right side (usually labeled 1), you have the "spam" emails. These are the unwanted advertising or junk emails.

The height of each bar shows how many emails there are in each category. In your data, it seems there are more "ham" emails (taller bar) than "spam" emails (shorter bar). This helps visualize the balance between desired and unwanted emails in your dataset.

Step 4: Text Vectorization with Count vectorizer

```
vectorizer = CountVectorizer()
spamham_countVectorizer = vectorizer.fit_transform(spam_df['text'])
```

```
#Print the feature names
print(vectorizer.get_feature_names_out())
```

```
['00' '000' '0000' ... 'zzn' 'zzncacst' 'zzzz']
```

- `vectorizer = CountVectorizer()`: This line creates an instance of the `CountVectorizer` class from scikit-learn. This class is used to convert text data into numerical features.

- `.fit_transform(spam_df['text'])`: This line applies the `fit_transform` method to the vectorizer object. It does two things:
- **Fit**: The vectorizer analyzes the text data in the 'text' column of `spam_df` to identify unique words (vocabulary). This process is called fitting.
- **Transform**: The vectorizer then converts each email in the 'text' column into a numerical feature vector. This vector represents the frequency (count) of each word in the vocabulary appearing in that email. The output is stored in a new sparse matrix called `spamham_countVectorizer`.
- `.get_feature_names_out()`: This line calls the `get_feature_names_out` method on the vectorizer object. This method returns the vocabulary (unique words) identified during the fitting process. These vocabulary words become the feature names in the transformed sparse matrix.

```
# Display the shape of the document-term matrix
print(spamham_countVectorizer.shape)
```

```
(5728, 37303)
```

It will reveal the number of emails (rows) and the number of unique words (features or columns) in your dataset after vectorization with `CountVectorizer`.

- **5728 (rows)**: This indicates the number of emails in your original dataset. There are likely 5728 emails that were vectorized using `CountVectorizer`.
- **37303 (columns)**: This represents the number of unique words (features) identified by `CountVectorizer` in the email text data. After transforming the text data, the resulting matrix has 37303 columns, each corresponding to a unique word found in the emails.

Step 5: Training the model

```
X = spamham_countVectorizer
y = spam_df['spam']
```

```
#Printing dimension of X and y
print(X.shape)
print(y.shape)
```

```
(5728, 37303)
(5728,)
```

- `X = spamham_countVectorizer`: This line assigns the previously created sparse matrix `spamham_countVectorizer` (containing the numerical features) to the variable `X`. This matrix will be used as the feature set for the model.
- `y = spam_df['spam']`: This line extracts the 'spam' column from the `spam_df` DataFrame and assigns it to the variable `y`. This column contains the labels (0 for ham and 1 for spam) which will be used as the target variable for the model.
- `.shape`: This method is called on both `X` and `y` to print their shapes. The shape of a matrix indicates the number of rows and columns.

Step 6: Splitting the dataset into Training and Testing datasets

```
#Splitting training-testing data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

It splits the data you prepared earlier (features `X` and labels `y`) into training and testing sets.

- `X_train`: This variable will store the training features (a subset of the original `X`).
- `X_test`: This variable will store the testing features (another subset of the original `X`).
- `y_train`: This variable will store the training labels (a subset of the original `y` corresponding to the training features in `X_train`).
- `y_test`: This variable will store the testing labels (a subset of the original `y` corresponding to the testing features in `X_test`).

`test_size=0.2`: This argument specifies the proportion of data to be used for the testing set. In this case, 0.2 represents 20%, so 80% of the data will be allocated to the training set and 20% to the testing set.

`random_state=42`: This argument sets a seed for the random number generator used to split the data. This ensures reproducibility - if you run the code again with the same `random_state`, you'll get the same split.

Step 7: Training a Naive Bayes classifier for spam email classification.

```
#Training Naive Bayes classifier
NB_classifier = MultinomialNB()
NB_classifier.fit(X_train, y_train)
```

▼ MultinomialNB

MultinomialNB()

- `NB_classifier = MultinomialNB()`: This line creates an instance of the `MultinomialNB` class from `scikit-learn`. This class implements the Multinomial Naive Bayes algorithm, a popular choice for text classification tasks.

- `.fit(X_train, y_train)`: This line trains the Naive Bayes classifier using the training data (`X_train` features and `y_train` labels). During training, the classifier learns the statistical relationship between the features (words) in emails and their corresponding spam/ham labels.

Step 8: Evaluating the model

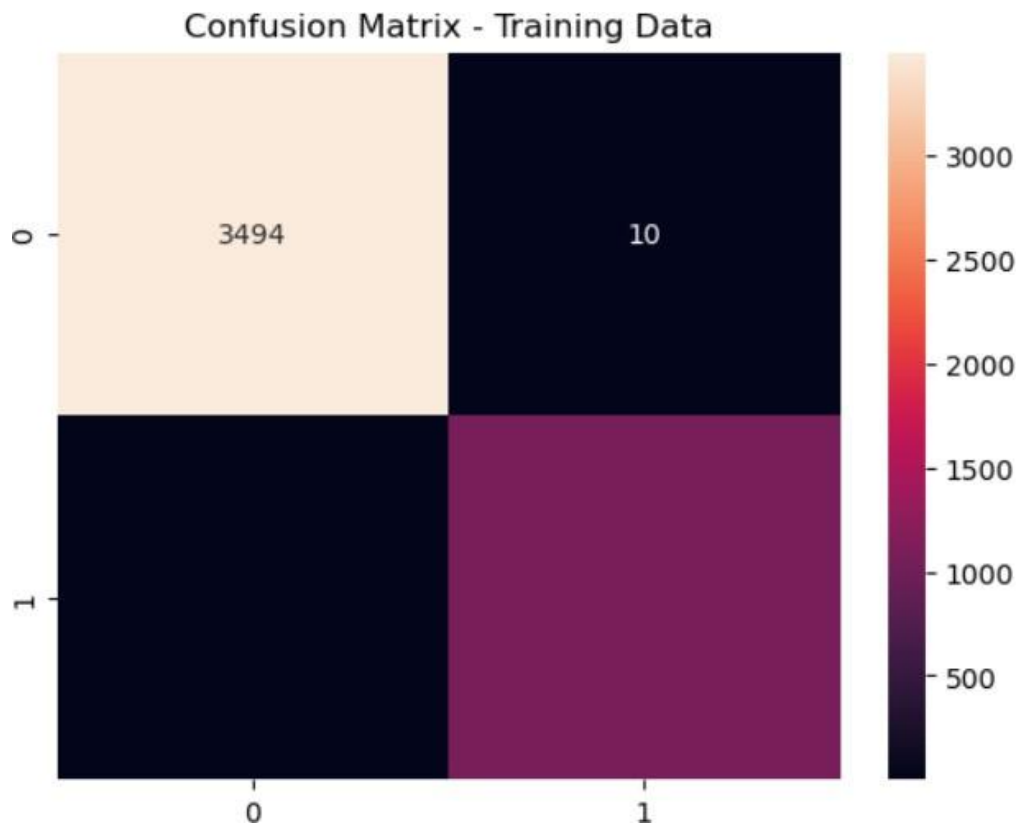
```
#Predicting on training data
y_predict_train = NB_classifier.predict(X_train)
```

```
#Confusion matrix for training data
cm_train = confusion_matrix(y_train, y_predict_train)
sns.heatmap(cm_train, annot=True, fmt='d')
plt.title('Confusion Matrix - Training Data')
plt.show()
```

- `y_predict_train = NB_classifier.predict(X_train)`: This line uses the trained Naive Bayes classifier (`NB_classifier`) to predict spam/ham labels for the emails in the training data (`X_train`). The predictions are stored in the `y_predict_train` variable.

Evaluating the Model on Training Data (using Confusion Matrix):

- `cm_train = confusion_matrix(y_train, y_predict_train)`: This line calculates the confusion matrix for the model's performance on the training data. The confusion matrix compares the actual labels (`y_train`) with the predicted labels (`y_predict_train`). It shows how many emails were correctly classified as spam (true positives - TP), ham (true negatives - TN), incorrectly classified as spam (false positives - FP), and incorrectly classified as ham (false negatives - FN).

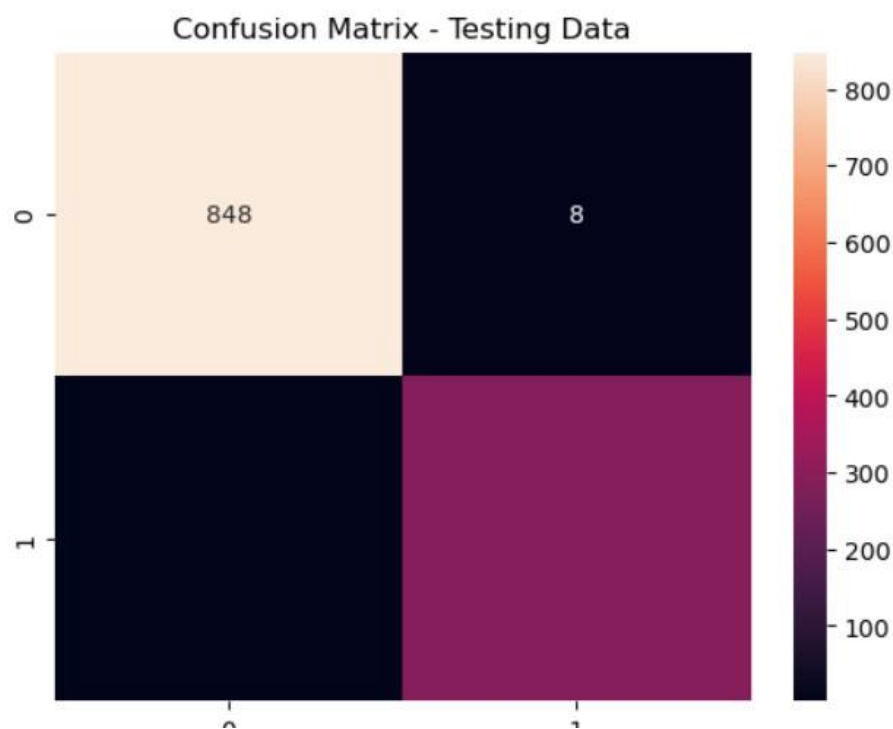


- `sns.heatmap(cm_train, annot=True, fmt='d')`: This line creates a heatmap using seaborn. The heatmap visually represents the values in the confusion matrix. Darker squares typically indicate higher values.
- `annot=True`: This argument specifies to annotate (display the actual numbers) within each square of the heatmap.
- `fmt='d'`: This argument specifies the format for the numbers displayed in the heatmap ('d' for integers).
- `plt.title('Confusion Matrix - Training Data')`: This line sets the title for the plot.
- `plt.show()`: This line displays the heatmap visualization you just created

For evaluating the performance on the Testing data

```
#Prediction on testing data
y_predict_test = NB_classifier.predict(X_test)
```

```
#Confusion matrix for testing data
cm_test = confusion_matrix(y_test, y_predict_test)
sns.heatmap(cm_test, annot=True, fmt='d')
plt.title('Confusion Matrix - Testing Data')
plt.show()
```



Step 9: Classification Report

```
# Print classification report for testing data
print(classification_report(y_test, y_predict_test))
```

	precision	recall	f1-score	support
0	1.00	0.99	0.99	856
1	0.97	0.99	0.98	290
accuracy			0.99	1146
macro avg	0.99	0.99	0.99	1146
weighted avg	0.99	0.99	0.99	1146

The classification report provides detailed metrics to evaluate the performance of the Naive Bayes classifier on the test set. Each metric is calculated for both classes: class 0 (ham) and class 1 (spam).

Class 0 (Ham)

- Precision: 1.00
 - Precision is the ratio of true positive predictions to the total predicted positives. For class 0, it means that 100% of the emails predicted as ham were actually ham.
- Recall: 0.99
 - Recall is the ratio of true positive predictions to the total actual positives. For class 0, it means that 99% of the actual ham emails were correctly predicted.
- F1-Score: 0.99
 - The F1-Score is the harmonic mean of precision and recall. For class 0, it is 0.99, indicating a balance between precision and recall.
- Support: 856
 - Support is the number of actual occurrences of the class in the test set. There are 856 actual ham emails.

Class 1 (Spam)

- Precision: 0.97
 - Precision for class 1 means that 97% of the emails predicted as spam were actually spam.
- Recall: 0.99
 - Recall for class 1 means that 99% of the actual spam emails were correctly predicted.
- F1-Score: 0.98
 - The F1-Score for class 1 is 0.98, indicating a good balance between precision and recall.
- Support: 290
 - There are 290 actual spam emails in the test set.

Overall Metrics

- Accuracy: 0.99
 - Accuracy is the ratio of correctly predicted instances to the total instances. An accuracy of 0.99 means that 99% of the emails were correctly classified as either spam or ham.
- Macro Average:
 - Precision: 0.99
 - The average precision considering both classes equally.

- Recall: 0.99
 - The average recall considering both classes equally.
- F1-Score: 0.99
 - The average F1-Score considering both classes equally.
- Weighted Average:
 - Precision: 0.99
 - The average precision weighted by the number of instances in each class.
 - Recall: 0.99
 - The average recall weighted by the number of instances in each class.
 - F1-Score: 0.99
 - The average F1-Score weighted by the number of instances in each class.

Test a Genuine email

```
sample_email = ["Congratulations! You have won a free iPhone. Click here to claim your prize now!"]

sample_vector = vectorizer.transform(sample_email)

prediction = NB_classifier.predict(sample_vector)

print(prediction)
```

Check the output

```
sample_email = ["Hi team, please find the project report attached. Let me know your feedback."]

sample_vector = vectorizer.transform(sample_email)

prediction = NB_classifier.predict(sample_vector)

print(prediction)
```

Practical Lesson 3.3: Decision Tree Algorithm for Predicting Customer Purchases

Overview

In this practical, we aim to predict whether a customer will make a purchase based on their age and estimated salary. We use the Decision Tree algorithm to classify customers into two categories: those who made a purchase (labeled as 1) and those who did not (labeled as 0). By visualizing the decision boundary generated by the algorithm, we gain insights into how age and estimated salary influence purchasing decisions. This classification can help businesses target their marketing efforts more effectively by identifying potential customers who are more likely to make a purchase.

Prerequisite

- Python3 should be configured.
- Anaconda Installer should be configured
- Good internet connection

Tasks

Step 1: Data Preprocessing

1. Import the libraries.

```
# importing libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

2. Import the dataset

```
#importing datasets
data_set= pd.read_csv('User_Data.csv')
```

3. Extracting the features and target from the dataset

```
#Extracting Independent and dependent Variable
x= data_set.iloc[:, [2,3]].values
y= data_set.iloc[:, 4].values
```

- It extracts the features from the dataset. Here, `iloc` is used to select data by index location. `[:, [2, 3]]` selects all rows and the columns at index locations 2 and 3. `.values` converts the selected data into a NumPy array.
- Then we extract the target variable from the dataset. It selects all rows and the column at index location 4. `.values` converts the selected data into a NumPy array.

4. Splitting the dataset into training and testing set

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size= 0.25, random_state=0)
```

It splits the features (**x**) and the target variable (**y**) into random train and test subsets. It takes several parameters:

- **x**: The features.
- **y**: The target variable.
- **test_size**: The proportion of the dataset to include in the test split. Here, it's set to 0.25, meaning 25% of the data will be used for testing.
- **random_state**: Controls the shuffling applied to the data before splitting. Passing an integer value ensures the reproducibility of the results.

5. Feature Extraction

```
#feature Scaling
from sklearn.preprocessing import StandardScaler
st_x = StandardScaler()
x_train = st_x.fit_transform(x_train)
x_test = st_x.transform(x_test)
```

- **st_x = StandardScaler()**: This creates an instance of the **StandardScaler** class, which will be used to scale the features.
- **x_train = st_x.fit_transform(x_train)**: This line fits the scaler to the training data (**x_train**) and then transforms it. The **fit_transform** method calculates the mean and standard deviation of each feature in the training set and scales the training data accordingly.
- **x_test = st_x.transform(x_test)**: This line uses the same scaler (**st_x**) to transform the test data (**x_test**). It's important to use the same scaler that was fitted to the training data to ensure consistency in scaling.

Step 2: Fitting a Decision-Tree algorithm to the Training set

Now we will fit the model to the training set. For this, we will import the **DecisionTreeClassifier** class from **sklearn.tree** library. Below is the code for it:

```
#Fitting Decision Tree classifier to the training set
from sklearn.tree import DecisionTreeClassifier
classifier = DecisionTreeClassifier(criterion='entropy', random_state=0)
classifier.fit(x_train, y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(criterion='entropy', random_state=0)
```

- **classifier = DecisionTreeClassifier(criterion='entropy', random_state=0)**: This creates an instance of the **DecisionTreeClassifier** class with the specified parameters:

- **criterion='entropy'**: This parameter specifies the criterion used to measure the quality of a split. In this case, it's set to 'entropy', which measures the amount of information gained by each split based on the entropy of the target variable.
- **random_state=0**: This parameter is used to initialize the random number generator, which ensures reproducibility of the results.
- **classifier.fit(x_train, y_train)**: This line trains the decision tree classifier on the training data (**x_train** and **y_train**). The **fit** method fits the model to the training data by learning the decision tree structure based on the features (**x_train**) and corresponding target labels (**y_train**).

Step 3: Predicting the test result

Now we will predict the test set result. We will create a new prediction vector **y_pred**. Below is the code for it:

```
#Predicting the test set result
y_pred= classifier.predict(x_test)
```

```
print(y_test)
```

```
[0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 0 0 0
 0 0 1 0 0 0 0 1 0 0 1 0 1 1 0 0 0 1 1 0 0 1 0 0 1 0 1 0 1 0 0 0 0 1 0 0 1
 0 0 0 0 1 1 1 0 0 0 1 1 0 1 1 0 0 1 0 0 0 1 0 1 1 1]
```

```
print(y_pred)
```

```
[0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 1 1 0 1 0 0 1 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0
 0 0 1 0 0 0 0 1 0 0 1 0 1 1 0 0 1 1 1 0 0 1 0 0 1 0 1 0 1 0 0 0 1 1 0 0 1
 0 0 0 0 1 1 1 1 0 0 1 0 0 1 1 0 0 1 0 0 0 1 0 1 1 1]
```

It uses the trained decision tree classifier (**classifier**) to predict the target labels for the test data (**x_test**). The **predict** method takes the test data (**x_test**) as input and returns the predicted labels (**y_pred**) based on the learned decision tree model.

Step 4: Test accuracy of the result (Creation of Confusion matrix)

```
#Creating the Confusion matrix
from sklearn.metrics import confusion_matrix
cm= confusion_matrix(y_test, y_pred)
print(cm)
```

```
[[62  6]
 [ 3 29]]
```

It calculates the confusion matrix using the actual target labels (**y_test**) and the predicted target labels (**y_pred**). The function **confusion_matrix** takes two arguments: the actual labels (**y_test**) and the predicted labels (**y_pred**). It compares these labels and constructs a confusion matrix that shows the counts of true positive, true negative, false positive, and false negative predictions.

we can see the confusion matrix, which has **6+3= 9 incorrect predictions** and **62+29=91 correct predictions**.

Practical Lesson 3.4: Simple Linear Regression for Predicting Salaries Based on Experience

Overview

Here we are taking a dataset that has two variables: salary (dependent variable) and experience (Independent variable). The goals of this problem is:

- We want to find out if there is any correlation between these two variables
- We will find the best fit line for the dataset.
- How the dependent variable is changing by changing the independent variable.

Prerequisite

- Python3 should be configured.
- Anaconda Installer should be configured
- Good internet connection

Tasks

1. The first step for creating the Simple Linear Regression model is data pre-processing. First, we will import the three important libraries, which will help us load the dataset, plot the graphs, and create the Simple Linear Regression model.

```
import numpy as nm
import matplotlib.pyplot as mtp
import pandas as pd
```

2. Next, we will load the dataset into our code:

```
data_set= pd.read_csv('Salary_Data.csv')
```

3. Now to use the first and last rows of the dataset you can use the head and tail method.

```
print(data_set.head(10))
print(data_set.tail(10))
```

	YearsExperience	Salary
0	1.1	39343.0
1	1.3	46205.0
2	1.5	37731.0
3	2.0	43525.0
4	2.2	39891.0
5	2.9	56642.0
6	3.0	60150.0
7	3.2	54445.0
8	3.2	64445.0
9	3.7	57189.0

	YearsExperience	Salary
20	6.8	91738.0
21	7.1	98273.0
22	7.9	101302.0
23	8.2	113812.0
24	8.7	109431.0
25	9.0	105582.0
26	9.5	116969.0
27	9.6	112635.0
28	10.3	122391.0
29	10.5	121872.0

3. Extract the features and the target from the dataset

```
x= data_set.iloc[:, :-1].values
y= data_set.iloc[:, 1].values
```

```
print(x)
print(y)
```

- `[:, :-1]`: This selects all rows (`:`) and all columns except the last one (`:-1`). Specifically:
 - `:` means all rows.
 - `:-1` means all columns except the last one. In this case, it selects the **YearsExperience** column.
- `.values`: This converts the selected data into a NumPy array.

[:, 1]: This selects all rows (:) and the second column (1). Specifically:

- : means all rows.
- 1 means the second column, which in this case is the Salary column.

.values: This converts the selected data into a NumPy array.

4. Next, we will split both variables into the test set and training set. We have 30 observations, so we will take 20 observations for the training set and 10 observations for the test set. We are splitting our dataset so that we can train our model using a training dataset and then test the model using a test dataset. The code for this is given below:

```
# Splitting the dataset into training and test set.  
from sklearn.model_selection import train_test_split  
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size= 1/3, random_state=0)
```

- **x_train**: Contains the training set of the independent variable (**YearsExperience**).
- **x_test**: Contains the test set of the independent variable (**YearsExperience**).
- **y_train**: Contains the training set of the dependent variable (**Salary**).
- **y_test**: Contains the test set of the dependent variable (**Salary**).

Parameters of train_test_split:

- **x**: The feature array (independent variable) containing the **YearsExperience** values.
- **y**: The target array (dependent variable) containing the **Salary** values.
- **test_size=1/3**: Specifies that 1/3 of the data should be allocated to the test set, and the remaining 2/3 should be allocated to the training set.
- **random_state=0**: Sets the random seed for reproducibility. This ensures that the data is split the same way every time the code is run.

By printing the variables, we will get x-test, x-train, and y-test, y-train dataset. Consider the below images:

```
print(x_test)  
print(y_test)
```

```
[[ 1.5]  
 [10.3]  
 [ 4.1]  
 [ 3.9]  
 [ 9.5]  
 [ 8.7]  
 [ 9.6]  
 [ 4. ]  
 [ 5.3]  
 [ 7.9]]  
[ 37731. 122391.  57081.  63218. 116969. 109431. 112635.  55794.  83088.  
 101302.]
```


Now in this step, we will visualize the training set result. To do so, we will use the `scatter()` function of the `pyplot` library, which we have already imported in the pre-processing step. The **scatter () function** will create a scatter plot of observations.

In the x-axis, we will plot the Years of Experience of employees and on the y-axis, salary of employees. In the function, we will pass the real values of training set, which means a year of experience `x_train`, training set of Salaries `y_train`, and color of the observations. Here we are taking a green color for the observation, but it can be any color as per the choice.

Now, we need to plot the regression line, so for this, we will use the **plot() function** of the `pyplot` library. In this function, we will pass the years of experience for training set, predicted salary for training set `x_pred`, and color of the line.

Next, we will give the title for the plot. So here, we will use the **title()** function of the `pyplot` library and pass the name ("Salary vs Experience (Training Dataset)").

After that, we will assign labels for x-axis and y-axis using **xlabel()** and **ylabel()** function.

Finally, we will represent all above things in a graph using `show()`. The code is given below:

```
mtp.scatter(x_train, y_train, color="green")
mtp.plot(x_train, x_pred, color="red")
mtp.title("Salary vs Experience (Training Dataset)")
mtp.xlabel("Years of Experience")
mtp.ylabel("Salary(In Rupees)")
mtp.show()
```



The good fit of the line can be observed by calculating the difference between actual values and predicted values. But as we can see in the above **plot**, **most of the observations are close to the regression line**, hence our model is good for the training set.

Step: 5. visualizing the Test set results:

In the previous step, we have visualized the performance of our model on the training set. Now, we will do the same for the Test set. The complete code will remain the same as the above code, except in this, we will use `x_test`, and `y_test` instead of `x_train` and `y_train`.

Here we are also changing the color of observations and regression line to differentiate between the two plots, but it is optional.

```
#visualizing the Test set results
mtp.scatter(x_test, y_test, color="blue")
mtp.plot(x_train, x_pred, color="red")
mtp.title("Salary vs Experience (Test Dataset)")
mtp.xlabel("Years of Experience")
mtp.ylabel("Salary(In Rupees)")
mtp.show()
```

Output:

By executing the above line of code, we will get the output as:



In the above plot, there are observations given by the blue color, and prediction is given by the red regression line. As we can see, most of the observations are close to the regression line, hence we can say our Simple Linear Regression is a good model and able to make good predictions.

Practical Lesson 3.5: Logistic Regression for Classification based on Age and Estimated Salary

Overview

There is a dataset which contains the information of various users obtained from the social networking sites. There is a car-making company that has recently launched a new SUV car. So the company wanted to check how many users from the dataset, wants to purchase the car.

For this problem, we will build a Machine Learning model using the Logistic regression algorithm. In this problem, we will predict the purchased variable (Dependent Variable) by using age and salary (Independent variables).

Prerequisite

- Python3 should be configured.
- Anaconda Installer should be configured
- Good internet connection

Tasks

Step 1: Data Preprocessing

In this step, we will pre-process/prepare the data so that we can use it in our code efficiently. It will be the same as we have done in other Data pre-processing topic. The code for this is given below:

```
#Data Pre-procesing Step
# importing libraries
import numpy as nm
import matplotlib.pyplot as mtp
import pandas as pd
```

For importing the dataset

```
#importing datasets
data_set= pd.read_csv('User_Data.csv')
```

```
print(data_set.head(10))
print(data_set.tail(10))
```

We have used the head and the tail method to print the first and the last rows of the dataset and below is the output

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0
5	15728773	Male	27	58000	0
6	15598044	Female	27	84000	0
7	15694829	Female	32	150000	1
8	15600575	Male	25	33000	0
9	15727311	Female	35	65000	0
	User ID	Gender	Age	EstimatedSalary	Purchased
390	15807837	Male	48	33000	1
391	15592570	Male	47	23000	1
392	15748589	Female	45	45000	1
393	15635893	Male	60	42000	1
394	15757632	Female	39	59000	0
395	15691863	Female	46	41000	1
396	15706071	Male	51	23000	1
397	15654296	Female	50	20000	1
398	15755018	Male	36	33000	0
399	15594041	Female	49	36000	1

Now, we will extract the dependent and independent variables from the given dataset. Below is the code for it:

```
#Extracting Independent and dependent Variable
x= data_set.iloc[:, [2,3]].values
y= data_set.iloc[:, 4].values
```

So, after executing this code, **x** will contain the independent variables(Age, estimated salary), and **y** (Purchased) will contain the dependent variable.

These variables can then be used for further analysis, such as model training and prediction.

Now we will split the dataset into a training set and test set. Below is the code for it:

```
# Splitting the dataset into training and test set.
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test= train_test_split(x, y, test_size= 0.25, random_state=0)
```

- **x**: The independent variables (features).
- **y**: The dependent variable (target).
- **test_size=0.25**: Specifies the proportion of the dataset to include in the test split. Here, 25% of the data will be used for testing.
- **random_state=0**: Sets the random seed for reproducibility. It ensures that the same random splits are generated each time the code is run, which is useful for reproducibility of results.

Testing set

```
print(x_test)
print(y_test)
```

```
[[ 30 87000]
 [ 38 50000]
 [ 35 75000]
 [ 30 79000]
 [ 35 50000]
 [ 27 20000]
 [ 31 15000]
 [ 30 44000]
 ...
]
[[0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 0 0 0
 0 0 1 0 0 0 0 1 0 0 1 0 1 1 0 0 0 1 1 0 0 1 0 0 1 0 1 0 1 0 0 0 0 1 0 0 1
 0 0 0 0 1 1 1 0 0 0 1 1 0 1 1 0 0 1 0 0 0 1 0 1 1 1]
```

Training set

```
print(x_train)
print(y_train)
```

```
[[ 44 39000]
 [ 32 120000]
 [ 38 50000]
 [ 32 135000]
 [ 52 21000]
 [ 53 104000]
 ...
]
[[0 1 0 1 1 1 0 0 0 0 0 0 1 1 1 0 1 0 0 1 0 1 0 1 0 0 1 1 1 1 0 1 0 1 0 0 1
 0 0 1 0 0 0 0 0 1 1 1 1 0 0 0 1 0 1 0 1 0 0 1 0 0 0 1 0 0 0 1 1 0 0 1 0 1
 1 1 0 0 1 1 0 0 1 1 0 1 0 0 1 1 0 1 1 1 0 0 0 0 0 1 0 0 1 1 1 1 1 0 1 1 0
 1 0 0 0 0 0 0 0 1 1 0 0 1 0 0 1 0 0 0 1 0 1 1 0 1 0 0 0 0 1 0 0 0 1 1 0 0
 0 0 1 0 1 0 0 0 1 0 0 0 0 1 1 1 0 0 0 0 0 0 0 1 1 1 1 0 1 0 0 0 0 0 1 0 0
 0 0 0 0 1 1 0 1 0 1 0 0 1 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 1 0 1 1 0 0 0 0 0
 0 1 1 0 0 0 0 1 0 0 0 0 1 0 1 0 1 0 0 0 1 0 0 0 1 0 1 0 0 0 0 0 1 1 0 0 0
 0 0 1 0 1 1 0 0 0 0 0 1 0 1 0 0 1 0 0 1 0 1 0 0 0 0 0 0 1 1 1 1 0 0 0 0 1
 0 0 0 0]
```

In logistic regression, we will do feature scaling because we want accurate result of predictions. Here we will only scale the independent variable because dependent variable have only 0 and 1 values. Below is the code for it:

```
#feature Scaling
from sklearn.preprocessing import StandardScaler
st_x= StandardScaler()
x_train= st_x.fit_transform(x_train)
x_test= st_x.transform(x_test)
```

- We have Created an instance of the **StandardScaler** class called **st_x**. This object will be used to scale the features.

- **st_x.fit_transform(x_train)**: Fits the scaler to the training data (**x_train**) and transforms it. This means that the mean and standard deviation are calculated from the training data, and then the training data is scaled accordingly. The scaled data is then assigned back to **x_train**.
- **st_x.transform(x_test)**: Uses the scaler fitted on the training data to transform the test data (**x_test**). This ensures that the same transformation is applied to both training and test data, maintaining consistency. The scaled test data is then assigned back to **x_test**.

2. Fitting Logistic Regression to the Training set:

We have well prepared our dataset, and now we will train the dataset using the training set. For providing training or fitting the model to the training set, we will import the **LogisticRegression** class of the **sklearn** library.

After importing the class, we will create a classifier object and use it to fit the model to the logistic regression. Below is the code for it:

```
#Fitting Logistic Regression to the training set
from sklearn.linear_model import LogisticRegression
classifier= LogisticRegression(random_state=0)
classifier.fit(x_train, y_train)
```

```
LogisticRegression
LogisticRegression(random_state=0)
```

- We have Created an instance of the **LogisticRegression** class called **classifier**. The parameter **random_state=0** sets the random seed for reproducibility of results.
- **classifier.fit(x_train, y_train)**: Fits the logistic regression model to the training data. The **fit** method learns the parameters of the logistic regression model using the training data (**x_train** and **y_train**). The independent variables are passed as **x_train**, and the corresponding target variable is passed as **y_train**.

3. Predicting the Test Result

Our model is well trained on the training set, so we will now predict the result by using test set data. Below is the code for it

```
#Predicting the test set result
y_pred= classifier.predict(x_test)
```

```
print(y_pred)
```

```
[0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 1 0 0 1 0 1 0 1 0 0 0 0 0 0 1 0 0 0 0
 0 0 1 0 0 0 0 1 0 0 1 0 1 1 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 1 0 0 0
 0 0 1 0 1 1 1 1 0 0 1 1 0 1 0 0 0 1 0 0 0 0 0 0 0 1 1]
```

We used the trained logistic regression model (**classifier**) to predict the target variable for the test set (**x_test**). It generates predictions based on the features in **x_test**.

4. Test Accuracy of the result

Now we will create the confusion matrix here to check the accuracy of the classification. To create it, we need to import the **confusion_matrix** function of the sklearn library. After importing the function, we will call it using a new variable **cm**.

```
#Creating the Confusion matrix
from sklearn.metrics import confusion_matrix
cm= confusion_matrix(y_test,y_pred)
print(cm)

[[65  3]
 [ 8 24]]
```

We can find the accuracy of the predicted result by interpreting the confusion matrix. By above output, we can interpret that $65+24= 89$ (Correct Output) and $8+3= 11$ (Incorrect Output).

5. Visualizing the training set result

Finally, we will visualize the training set result. To visualize the result, we will use **ListedColormap** class of matplotlib library. Below is the code for it:

```
from matplotlib.colors import ListedColormap

x_set, y_set = x_train, y_train
x1, x2 = nm.meshgrid(nm.arange(start=x_set[:, 0].min() - 1, stop=x_set[:, 0].max() + 1, step=0.01),
                    nm.arange(start=x_set[:, 1].min() - 1, stop=x_set[:, 1].max() + 1, step=0.01))

# Create a ListedColormap with valid color values
colors = ['#FF00FF', '#008000'] # Purple and Green

mtp.contourf(x1, x2, classifier.predict(nm.array([x1.ravel(), x2.ravel()]).T).reshape(x1.shape),
             alpha=0.75, cmap=ListedColormap(colors))

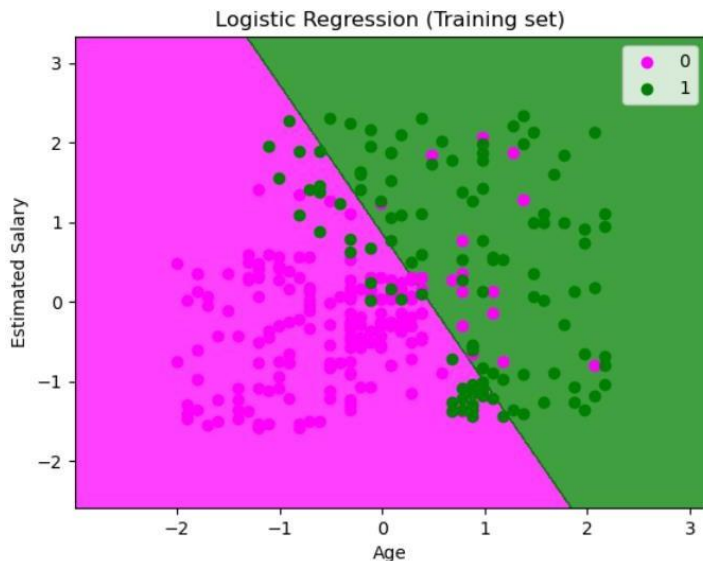
mtp.xlim(x1.min(), x1.max())
mtp.ylim(x2.min(), x2.max())

mtp.title('Logistic Regression (Training set)')
mtp.xlabel('Age')
mtp.ylabel('Estimated Salary')
mtp.legend()
mtp.show()
```

- Imports the **ListedColormap** class from the **matplotlib.colors** module. **ListedColormap** is used to create custom colormaps for contour plots.
- Assigns the training set data (**x_train, y_train**) to **x_set** and **y_set**, respectively.
- Creates a meshgrid of points spanning the range of the features in the training set. This grid will be used to visualize the decision boundary.
- Defines the colors for the contour plot. In this case, purple (**#FF00FF**) and green (**#008000**) are chosen.
- Generates a filled contour plot representing the decision boundary of the logistic regression model on the training set. The **classifier.predict()** method predicts the class labels for all points in the meshgrid, and the result is reshaped to match the meshgrid shape. The **contourf** function fills the contour plot based on these predictions.
- Sets the limits of the plot to match the extent of the meshgrid.

- Sets the title and labels for the plot.
- Displays the plot.

Output: By executing the above code, we will get the below output



The graph can be explained in the below points:

- In the above graph, we can see that there are some **Green points** within the green region and **Purple points** within the purple region.
- All these data points are the observation points from the training set, which shows the result for purchased variables.
- This graph is made by using two independent variables i.e., **Age on the x-axis** and **Estimated salary on the y-axis**.
- The **purple point observations** are for which purchased (dependent variable) is probably 0, i.e., users who did not purchase the SUV car.
- The **green point observations** are for which purchased (dependent variable) is probably 1 means user who purchased the SUV car.
- We can also estimate from the graph that the users who are younger with low salary, did not purchase the car, whereas older users with high estimated salary purchased the car.
- But there are some purple points in the green region (Buying the car) and some green points in the purple region (Not buying the car). So we can say that younger users with a high estimated salary purchased the car, whereas an older user with a low estimated salary did not purchase the car.

The goal of the classifier:

We have successfully visualized the training set result for the logistic regression, and our goal for this classification is to divide the users who purchased the SUV car and who did not purchase the car. So from the output graph, we can clearly see the two regions (Purple and Green) with the observation points. The Purple region is for those users who didn't buy the car, and Green Region is for those users who purchased the car.

Linear Classifier:

As we can see from the graph, the classifier is a Straight line or linear in nature as we have used the Linear model for Logistic Regression. In further topics, we will learn for non-linear Classifiers.

Visualizing the test set result:

Our model is well trained using the training dataset. Now, we will visualize the result for new observations (Test set). The code for the test set will remain same as above except that here we will use **x_test** and **y_test** instead of **x_train** and **y_train**. Below is the code for it:

```
# Visualization of the test set result
from matplotlib.colors import ListedColormap

# Define RGB tuples for the colors
colors = [(0.5, 0, 0.5), (0, 0.5, 0)]

# Creating a meshgrid of all points in the feature space
x_set, y_set = x_test, y_test
x1, x2 = nm.meshgrid(nm.arange(start=x_set[:, 0].min() - 1, stop=x_set[:, 0].max() + 1, step=0.01),
                     nm.arange(start=x_set[:, 1].min() - 1, stop=x_set[:, 1].max() + 1, step=0.01))

# Plotting the decision boundaries (contour plot) using the classifier's predictions
mtp.contourf(x1, x2, classifier.predict(nm.array([x1.ravel(), x2.ravel()]).T).reshape(x1.shape),
             alpha=0.75, cmap=ListedColormap(colors))

# Setting the limits of the plot
mtp.xlim(x1.min(), x1.max())
mtp.ylim(x2.min(), x2.max())

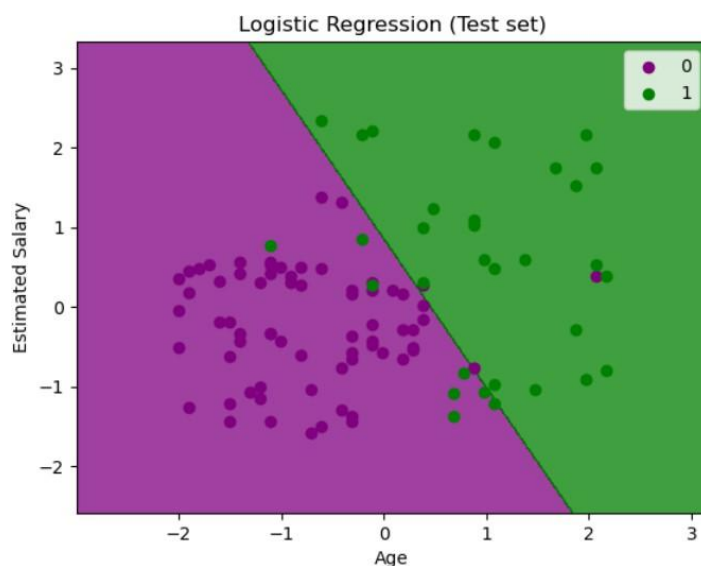
for i, j in enumerate(nm.unique(y_set)):
    mtp.scatter(x_set[y_set == j, 0], x_set[y_set == j, 1],
               color=ListedColormap(colors)(i), label=j)

# Adding title and labels
mtp.title('Logistic Regression (Test set)')
mtp.xlabel('Age')
mtp.ylabel('Estimated Salary')

# Adding Legend
mtp.legend()

# Displaying the plot
mtp.show()
```

Output



The above graph shows the test set result. As we can see, the graph is divided into two regions (Purple and Green). And Green observations are in the green region, and Purple observations are in the purple region. So we can say it is a good prediction and model. Some of the green and purple data points are in different regions, which can be ignored as we have already calculated this error using the confusion matrix (11 Incorrect output).

Hence our model is pretty good and ready to make new predictions for this classification problem.

Practical Lesson 3.6: K-Means Clustering using the Elbow Method

Overview

We have a dataset of Mall_Customers, which is the data of customers who visit the mall and spend there. In the given dataset, we have Customer_Id, Gender, Age, Annual Income (\$), and Spending Score (which is the calculated value of how much a customer has spent in the mall, the more the value, the more he has spent). From this dataset, we need to calculate some patterns, as it is an unsupervised method, so we don't know what to calculate exactly.

Prerequisite

- Python3 should be configured.
- Anaconda Installer should be configured
- Good internet connection

Tasks

Step-1: Data pre-processing Step

The first step will be the data pre-processing, as we did in our earlier topics of Regression and Classification. But for the clustering problem, it will be different from other models. Let's discuss it:

Importing Libraries

As we did in previous topics, firstly, we will import the libraries for our model, which is part of data pre-processing. The code is given below:

```
# importing libraries
import numpy as nm
import matplotlib.pyplot as mtp
import pandas as pd
```

In the above code, the numpy we have imported for the performing mathematics calculation, matplotlib is for plotting the graph, and pandas are for managing the dataset

Importing the Dataset:

Next, we will import the dataset that we need to use. So here, we are using the Mall_Customer_data.csv dataset. It can be imported using the below code:

```
# Importing the dataset
dataset = pd.read_csv('Mall_Customers.csv')
```

To view the dataset, use the head and the tail method

```
print(dataset.head(10))
print(dataset.tail(10))
```

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40
5	6	Female	22	17	76
6	7	Female	35	18	6
7	8	Female	23	18	94
8	9	Male	64	19	3
9	10	Female	30	19	72
190	191	Female	34	103	23
191	192	Female	32	103	69
192	193	Male	33	113	8
193	194	Female	38	113	91
194	195	Female	47	120	16
195	196	Female	35	120	79
196	197	Female	45	126	28
197	198	Male	32	126	74
198	199	Male	32	137	18
199	200	Male	30	137	83

From the above dataset, we need to find some patterns in it.

Extracting Independent Variables

Here we don't need any dependent variable for data pre-processing step as it is a clustering problem, and we have no idea about what to determine. So we will just add a line of code for the matrix of features.

```
x = dataset.iloc[:, [3, 4]].values

print(x)
```

So here we have extracted the Annual income feature and the Spending score and below is the output.

```
[ [ 15  39]
  [ 15  81]
  [ 16   6]
  [ 16  77]
  [ 17  40]
  [ 17  76]
  [ 18   6]
  [ 18  94]
  [ 19   3]
  [ 19  72]
  [ 19  14]
  [ 19  99]
  [ 20  15]
```

Step-2: Finding the optimal number of clusters using the elbow method

In the second step, we will try to find the optimal number of clusters for our clustering problem. So, as discussed above, here we are going to use the elbow method for this purpose.

As we know, the elbow method uses the WCSS concept to draw the plot by plotting WCSS values on the Y-axis and the number of clusters on the X-axis. So we are going to calculate the value for WCSS for different k values ranging from 1 to 10. Below is the code for it:

```

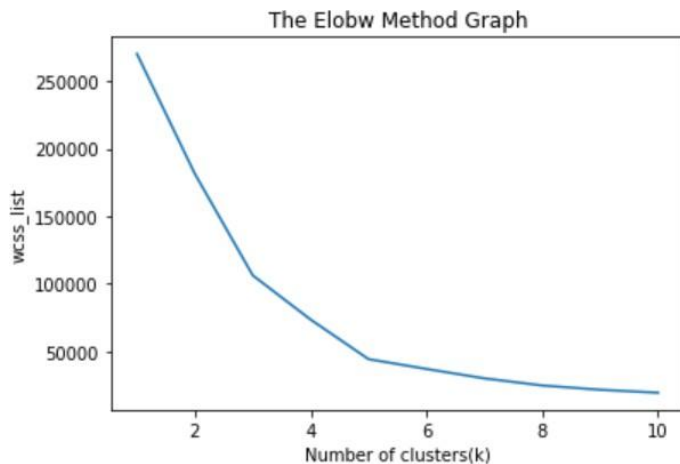
#finding optimal number of clusters using the elbow method
from sklearn.cluster import KMeans
wcss_list= [] #Initializing the list for the values of WCSS

#Using for loop for iterations from 1 to 10.
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state= 42)
    kmeans.fit(x)
    wcss_list.append(kmeans.inertia_)
mtp.plot(range(1, 11), wcss_list)
mtp.title('The Elbow Method Graph')
mtp.xlabel('Number of clusters(k)')
mtp.ylabel('wcss_list')
mtp.show()

```

- Imported the KMeans class, which will be used to create and train K-means clustering models.
- Then we created an empty list **wcss_list** which will store the within-cluster sum of squares (WCSS) values for different numbers of clusters.
- Then we used a for loop that iterates over the number of clusters from 1 to 10.
- For each iteration, this line creates an instance of the KMeans class with **i** clusters. **init='k-means++'** is a method for initialization to speed up convergence. **random_state=42** ensures reproducibility.
- Next we train the KMeans model on the dataset **x**.
- After fitting the model, this line calculates the WCSS for the current number of clusters and appends it to **wcss_list**. The WCSS is stored in the **inertia_** attribute of the fitted KMeans model.
- **The plot function** plots the number of clusters (1 to 10) on the x-axis and the corresponding WCSS values on the y-axis.
- **Also** sets the title of the plot to "The Elbow Method Graph".
- Used the xlabel which labels the x-axis as "Number of clusters(k)".
- Used the ylabel which labels the y-axis as "wcss_list".
- Show() is used to displays the plot.

After executing the above code, we will get the below output:



From the above plot, we can see the elbow point is at 5. So the number of clusters here will be 5.

Step- 3: Training the K-means algorithm on the training dataset

As we have got the number of clusters, so we can now train the model on the dataset.

To train the model, we will use the same two lines of code as we have used in the above section, but here instead of using `i`, we will use 5, as we know there are 5 clusters that need to be formed. The code is given below:

```
#training the K-means model on a dataset
kmeans = KMeans(n_clusters=5, init='k-means++', random_state= 42)
y_predict= kmeans.fit_predict(x)
```

The first line is the same as above for creating the object of KMeans class.

In the second line of code, we have created the dependent variable **y_predict** to train the model.

Step-4: Visualizing the Clusters

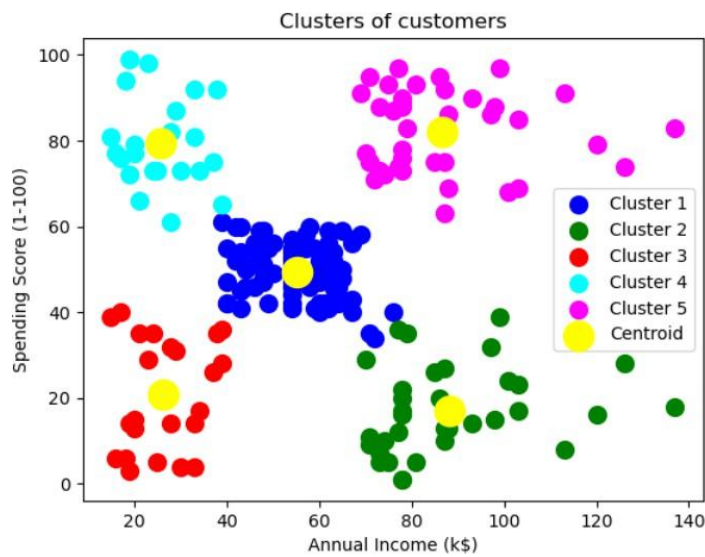
The last step is to visualize the clusters. As we have 5 clusters for our model, so we will visualize each cluster one by one.

To visualize the clusters will use scatter plot using `mtp.scatter()` function of matplotlib.

```
#visulaizing the clusters
mtp.scatter(x[y_predict == 0, 0], x[y_predict == 0, 1], s = 100, c = 'blue', label = 'Cluster 1') #for first cluster
mtp.scatter(x[y_predict == 1, 0], x[y_predict == 1, 1], s = 100, c = 'green', label = 'Cluster 2') #for second cluster
mtp.scatter(x[y_predict== 2, 0], x[y_predict == 2, 1], s = 100, c = 'red', label = 'Cluster 3') #for third cluster
mtp.scatter(x[y_predict == 3, 0], x[y_predict == 3, 1], s = 100, c = 'cyan', label = 'Cluster 4') #for fourth cluster
mtp.scatter(x[y_predict == 4, 0], x[y_predict == 4, 1], s = 100, c = 'magenta', label = 'Cluster 5') #for fifth cluster
mtp.scatter(kmeans.cluster_centers[:, 0], kmeans.cluster_centers[:, 1], s = 300, c = 'yellow', label = 'Centroid')
mtp.title('Clusters of customers')
mtp.xlabel('Annual Income (k$)')
mtp.ylabel('Spending Score (1-100)')
mtp.legend()
mtp.show() |
```

In above lines of code, we have written code for each clusters, ranging from 1 to 5. The first coordinate of the `mtp.scatter`, i.e., `x[y_predict == 0, 0]` containing the x value for the showing the matrix of features values, and the `y_predict` is ranging from 0 to 1.

Output:



The output image is clearly showing the five different clusters with different colors. The clusters are formed between two parameters of the dataset; Annual income of customer and Spending. We can change the colors and labels as per the requirement or choice. We can also observe some points from the above patterns, which are given below:

- Cluster1 shows the customers with average salary and average spending so we can categorize these customers as
- Cluster2 shows the customer has a high income but low spending, so we can categorize them as careful.
- Cluster3 shows the low income and also low spending so they can be categorized as sensible.
- Cluster4 shows the customers with low income with very high spending so they can be categorized as careless.
- Cluster5 shows the customers with high income and high spending so they can be categorized as target, and these customers can be the most profitable customers for the mall owner.

Practical Lesson 4: Text Preprocessing, Vectorization, and Sentiment Analysis Using Naive Bayes

Overview

In this activity, participants will learn the fundamental concepts of Natural Language Processing (NLP) and how textual data can be prepared for machine learning. Starting with raw text, they will perform essential preprocessing techniques such as lowercasing, tokenization, punctuation removal, stopwords removal, stemming, and lemmatization. Participants will then explore how text is transformed into numerical representations using Bag of Words (CountVectorizer) and TF-IDF Vectorization. Finally, they will build a simple Sentiment Analysis model using the Naive Bayes algorithm to classify text as positive or negative and test the model with custom inputs. This hands-on activity provides a complete introduction to the NLP pipeline, from text preprocessing to machine learning-based text classification.

Import Required Libraries

```
import nltk
import string
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
```

This cell imports the libraries required for performing Natural Language Processing (NLP) tasks. Before any text can be analyzed, it must be cleaned and transformed into a format that a machine learning model can understand. These libraries provide the tools needed for tokenization, stopwords removal, and text preprocessing.

Download Required NLTK Resources

```
nltk.download('punkt')
nltk.download('stopwords')
```

NLTK contains many resources that are not installed by default. This cell downloads the datasets required for tokenization and stopwords removal.

The Punkt package contains pretrained tokenization rules used for splitting text into words and sentences.

Stopwords Dataset: This downloads a predefined list of stopwords for various languages.

Cell 2: Create Sample Text

```
text = "Natural Language Processing is a branch of Artificial Intelligence. It helps computers understand human language."
print(text)
```

This cell creates a sample text document that will be used throughout the activity to demonstrate various NLP preprocessing techniques.

In real-world NLP applications, text can come from:

- Customer reviews
- Emails
- Chat messages
- News articles
- Social media posts
- Research papers

For simplicity, a single sentence is used as the input.

Cell 3: Convert Text to Lowercase

```
text_lower = text.lower()
print(text_lower)
```

This cell converts all characters in the text to lowercase using Python's built-in `lower()` function. Lowercasing is one of the most common text normalization techniques in NLP because it ensures that words with different capitalizations are treated as the same word.

For example, the words:

NLP

nlp

Nlp

have the same meaning for humans, but a computer would treat them as three different words. Converting all text to lowercase helps reduce the vocabulary size and improves consistency during analysis.

Cell 4: Tokenization

```
tokens = word_tokenize(text_lower)
print(tokens)
```

Tokenization is the process of breaking a sentence or document into smaller units called tokens. These tokens are usually individual words, punctuation marks, or symbols.

Machine learning models cannot directly understand long text strings. Therefore, text must first be divided into manageable components before further processing can be performed.

```
from nltk.tokenize import word_tokenize

tokens = word_tokenize(text_lower)

print(tokens)

['natural', 'language', 'processing', 'is', 'a', 'branch', 'of', 'artificial', 'intelligence', '.', 'it', 'helps', 'computers', 'understand', 'human', 'language', '.']
```

Each element in the list is called a token.

For example:

natural

language

processing

are individual word tokens.

Notice that punctuation is also treated as a token:

.'

This happens because the tokenizer recognizes punctuation as separate entities.

Cell 5: Remove Punctuation

```
import string

tokens_without_punctuation = []

for word in tokens:
    if word not in string.punctuation:
        tokens_without_punctuation.append(word)

print(tokens_without_punctuation)
```

This cell removes punctuation marks from the tokenized text. Punctuation symbols generally do not contribute significant meaning in many NLP applications and can introduce unnecessary noise into the dataset.

- The string module contains predefined constants, including all punctuation characters.
- `string.punctuation`
- `tokens_without_punctuation = []`
- This list will store only the valid words after punctuation removal.
- `for word in tokens:` The loop examines every token individually.
- `if word not in string.punctuation:`
- If the token is not a punctuation symbol, it is kept.
- `tokens_without_punctuation.append(word)`
- The valid token is added to the cleaned list.

Cell 6: Remove Stopwords

```
from nltk.corpus import stopwords

stop_words = set(stopwords.words('english'))

clean_tokens = []

for word in tokens_without_punctuation:
    if word not in stop_words:
        clean_tokens.append(word)

print(clean_tokens)
```

This cell removes stopwords from the text. Stopwords are commonly occurring words that usually provide little semantic meaning and can be safely removed in many NLP tasks.

Step 1: Load Stopwords

```
stop_words = set(stopwords.words('english'))
```

This retrieves the English stopwords list from NLTK.

Using a set improves lookup speed because membership testing is faster.

Step 2: Create Empty List

```
clean_tokens = []
```

This will store meaningful words after stopwords removal.

Step 3: Iterate Through Tokens

```
for word in tokens_without_punctuation:
```

Each token is checked individually.

Step 4: Remove Stopwords

```
if word not in stop_words:
```

Only words that are not stopwords are retained.

```

from nltk.corpus import stopwords

stop_words = set(stopwords.words('english'))

clean_tokens = []

for word in tokens_without_punctuation:
    if word not in stop_words:
        clean_tokens.append(word)

print(clean_tokens)

['natural', 'language', 'processing', 'branch', 'artificial', 'intelligence', 'helps', 'computers', 'understand', 'human', 'language']

```

Join Tokens Back into Text

```

clean_text = " ".join(clean_tokens)
print(clean_text)

```

After preprocessing, the text exists as a list of individual tokens. In many situations, it is useful to reconstruct the cleaned text into a readable sentence.

The join() function combines all words into a single string separated by spaces.

```
" ".join(clean_tokens)
```

The space " " acts as a separator between words.

Example:

```
["I", "love", "NLP"]
```

becomes

```
I love NLP
```

```

clean_text = " ".join(clean_tokens)

print(clean_text)

natural language processing branch artificial intelligence helps computers understand human language

```

Create a Dataset for Sentiment Analysis

```
import pandas as pd
```

```

data = {
    "text": [
        "I love this product",
        "This is a bad product",
        "I am very happy with the service",
        "The experience was terrible",
        "This is excellent",
        "I hate this item"
    ],
    "sentiment": [
        "positive",
        "negative",
        "positive",
        "negative",
        "positive",
        "negative"
    ]
}

```

```

df = pd.DataFrame(data)
print(df)

```

After performing text preprocessing on a single sentence, the next step is to create a dataset that can be used for machine learning. Machine learning algorithms require multiple examples to learn patterns and relationships between inputs and outputs. In this cell, a small sentiment analysis dataset is created manually using customer-review-like sentences and their corresponding sentiment labels.

The dataset contains two columns:

- **text** → Contains the review or opinion.
- **sentiment** → Contains the target label (positive or negative).

This is a supervised learning dataset because every text sample already has a known sentiment label.

A dictionary named `data` is then created containing two keys: `text` and `sentiment`. The `text` key stores a collection of sample customer review sentences, while the `sentiment` key stores the corresponding labels indicating whether each review expresses a positive or negative opinion.

The dictionary is then converted into a Pandas DataFrame using the `pd.DataFrame()` function. A DataFrame is a tabular data structure consisting of rows and columns, similar to an Excel spreadsheet or a database table. Converting the dictionary into a DataFrame makes the data easier to view, manipulate, and prepare for machine learning tasks.

This dataset represents a simple example of a supervised learning problem because every text sample already has a known output label. During training, the machine learning model will analyze the words present in the text and learn patterns associated with positive and negative sentiments. Words such as *love*, *happy*, and *excellent* are likely to be associated with positive sentiment, while words such as *bad*, *hate*, and *terrible* are likely to be associated with negative sentiment.

CountVectorizer (Bag of Words)

```
from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(df["text"])

print(vectorizer.get_feature_names_out())
```

- In this cell, the textual data is converted into a numerical format that can be understood by machine learning algorithms. Computers cannot directly process words and sentences; they can only work with numbers. To solve this problem, the `CountVectorizer` class from Scikit-learn is used.
- `CountVectorizer` implements a technique known as **Bag of Words (BoW)**, which converts text into numerical vectors based on word frequencies. The Bag of Words model focuses on the occurrence of words in a document while ignoring grammar and word order.
- First, the `CountVectorizer` class is imported from the `sklearn.feature_extraction.text` module. Then, an object named `vectorizer` is created. This object will be responsible for learning the vocabulary from the text dataset and converting the text into numerical features.
- The statement `vectorizer.fit_transform(df["text"])` performs two operations simultaneously. The `fit()` step scans all the text documents and identifies every unique word present in the dataset, thereby building a vocabulary. The `transform()` step then converts each document into a numerical vector by counting how many times each vocabulary word appears in that document. The resulting numerical representation is stored in the variable `X`.
- Finally, the method `get_feature_names_out()` is used to display all the unique words that

CountVectorizer has learned from the dataset. These words become the features or columns of the machine learning dataset. Each feature represents one unique word, and later each document will be represented by numerical values indicating the occurrence of these words.

- This step is extremely important because it serves as the bridge between natural language and machine learning, transforming unstructured text into structured numerical data.
- For example, if the sentence is "**I love this product**", CountVectorizer identifies words such as *love*, *this*, and *product*. During vectorization, the sentence will be represented as numerical values indicating the presence or absence of these words. Once all sentences are converted into vectors, they can be provided as input to machine learning algorithms such as Naive Bayes, Logistic Regression, or Support Vector Machines for text classification tasks.

```
from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(df["text"])

print(vectorizer.get_feature_names_out())

['am' 'bad' 'excellent' 'experience' 'happy' 'hate' 'is' 'item' 'love'
 'product' 'service' 'terrible' 'the' 'this' 'very' 'was' 'with']
```

View the Numerical Matrix

```
print(X.toarray())
```

After CountVectorizer converts text into numerical form, the result is stored as a sparse matrix. This cell converts that matrix into a regular array for visualization.

```
print(X.toarray())

[[0 0 0 0 0 0 0 0 1 1 0 0 0 1 0 0 0]
 [0 1 0 0 0 0 1 0 0 1 0 0 0 1 0 0 0]
 [1 0 0 0 1 0 0 0 0 0 0 1 0 1 0 1 0]
 [0 0 0 1 0 0 0 0 0 0 0 0 1 1 0 0 1]
 [0 0 1 0 0 0 1 0 0 0 0 0 0 0 1 0 0]
 [0 0 0 0 0 1 0 1 0 0 0 0 0 0 1 0 0]]
```

Most words do not appear in every sentence.

Therefore many values are:

0

To save memory, CountVectorizer stores data in sparse format.

X.toarray()

Converts the sparse matrix into a normal numerical matrix.

Expected Output

```
[[0 0 0 0 0 0 0 1 1 0 0 0 1 0 0 0]
 [0 1 0 0 0 1 0 0 1 0 0 0 1 0 0 0]
 [1 0 0 1 0 0 0 0 0 0 0 1 0 1 0 1]
 ...
 ]
```

Check Matrix Dimensions

```
print(X.shape)
```

The shape attribute provides information about the dimensions of the feature matrix.

```
print(X.shape)
```

```
(6, 17)
```

6 Rows

Represents:

6 sentences/documents

17 Columns

Represents:

17 unique words

identified by CountVectorizer.

Create the Target Variable

```
y = df["sentiment"]
```

```
print(y)
```

- In supervised machine learning, the dataset is divided into two main components: features and target variables. The features contain the input data used by the model to learn patterns, while the target variable contains the correct output that the model is expected to predict.
- In this activity, the feature matrix X contains the numerical representation of the text documents, whereas the sentiment column contains the labels indicating whether each review is positive or negative. This cell extracts the sentiment column from the DataFrame and stores it in a variable named y, which will serve as the target variable during model training.
- The statement `df["sentiment"]` selects the sentiment column from the DataFrame. This column contains the classification labels associated with each text document. These labels represent the ground truth that the model will learn from. When the model is trained, it will analyze the relationship between the word patterns present in X and the corresponding sentiment labels stored in y

```
y = df["sentiment"]
```

```
print(y)
```

```
0    positive
```

```
1    negative
```

```
2    positive
```

```
3    negative
```

```
4    positive
```

```
5    negative
```

```
Name: sentiment, dtype: object
```


Train the Naive Bayes Model

```
from sklearn.naive_bayes import MultinomialNB
model = MultinomialNB()
model.fit(X, y)
```

- In this cell, a machine learning model is created and trained using the numerical text features generated by CountVectorizer. The algorithm used is **Multinomial Naive Bayes**, which is one of the most widely used classification algorithms for text analytics and Natural Language Processing tasks.
- The MultinomialNB class is imported from the sklearn.naive_bayes module. An instance of the model is then created and stored in the variable model. The statement model.fit(X, y) trains the classifier using the feature matrix X and the target labels y.
- During training, the model analyzes the frequency of words associated with positive and negative sentiments. For example, words such as *love*, *happy*, and *excellent* may frequently appear in positive reviews, while words such as *bad*, *terrible*, and *hate* may frequently appear in negative reviews.

Test the Model with New Text

```
new_text = ["I love the service"]
new_text_vectorized = vectorizer.transform(new_text)
prediction = model.predict(new_text_vectorized)
print(prediction)
```

- After training the Naive Bayes classifier, the next step is to test whether the model can correctly classify new text that it has never seen before. In this cell, a new sentence, "**I love the service**", is provided as input.
- Since machine learning models cannot directly understand raw text, the sentence must first be converted into the same numerical format that was used during training. This is achieved using the transform() method of the previously trained CountVectorizer object. The resulting numerical representation is then supplied to the Naive Bayes model, which predicts the sentiment associated with the sentence.
- The statement vectorizer.transform(new_text) converts the input sentence into a Bag of Words vector using the same vocabulary learned during training. It is important to use transform() instead of fit_transform() because the vocabulary has already been learned from the training data. Using fit_transform() again would create a new vocabulary and make the prediction inconsistent with the training process.
- Once the sentence has been transformed into numerical features, the statement model.predict() uses the learned statistical relationships between words and sentiments to predict whether the sentence expresses a positive or negative opinion.
- The model examines the words present in the sentence, such as **love** and **service**, and compares them to the patterns it learned during training. Since words like *love* are strongly associated with positive sentiment in the training data, the classifier is expected to predict that the sentence expresses a positive opinion.

```

new_text = ["I love the service"]

new_text_vectorized = vectorizer.transform(new_text)

prediction = model.predict(new_text_vectorized)

print(prediction)

['positive']

```

Test the Model with a Negative Sentence

```

new_text = ["This product is terrible"]
new_text_vectorized = vectorizer.transform(new_text)
prediction = model.predict(new_text_vectorized)
print(prediction)

```

```

new_text = ["This product is terrible"]

new_text_vectorized = vectorizer.transform(new_text)

prediction = model.predict(new_text_vectorized)

print(prediction)

['negative']

```

Display the Bag of Words Matrix as a DataFrame

```

bow_df = pd.DataFrame(
    X.toarray(),
    columns=vectorizer.get_feature_names_out()
)
print(bow_df)

```

- Although the numerical matrix produced by CountVectorizer contains all the required information, it can be difficult to interpret because it is displayed as a series of numbers. In this cell, the Bag of Words matrix is converted into a Pandas DataFrame to make it more readable and easier to understand.
- Each row represents a document, and each column represents a vocabulary word learned by CountVectorizer. The values within the table indicate how many times a particular word appears in a given document.
- The statement `X.toarray()` converts the sparse matrix into a regular numerical array, while `vectorizer.get_feature_names_out()` retrieves the vocabulary words that will serve as column names. By combining these two components in a Pandas DataFrame, the numerical representation becomes much more intuitive. Instead of seeing anonymous columns represented by index positions, users can directly observe which words correspond to which numerical values.

	am	bad	excellent	experience	happy	hate	is	item	love	product	\
0	0	0	0	0	0	0	0	0	1	1	
1	0	1	0	0	0	0	1	0	0	1	
2	1	0	0	0	1	0	0	0	0	0	
3	0	0	0	1	0	0	0	0	0	0	
4	0	0	1	0	0	0	1	0	0	0	
5	0	0	0	0	0	1	0	1	0	0	

	service	terrible	the	this	very	was	with
0	0	0	0	1	0	0	0
1	0	0	0	1	0	0	0
2	1	0	1	0	1	0	1
3	0	1	1	0	0	1	0
4	0	0	0	1	0	0	0
5	0	0	0	1	0	0	0

Each row corresponds to one document from the dataset, while each column corresponds to a unique vocabulary word. A value of **1** indicates that the word appears in the document, while a value of **0** indicates that the word is absent.

For example, in the first document:

I love this product

the words:

love

product

this

have values of **1**, while all other vocabulary words have values of **0**.

This table provides a clear representation of how text is transformed into numerical features before being supplied to a machine learning algorithm.

TF-IDF Vectorization

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf_vectorizer = TfidfVectorizer()
X_tfidf = tfidf_vectorizer.fit_transform(df["text"])
print(tfidf_vectorizer.get_feature_names_out())
```

- In this cell, the text data is converted into numerical form using TF-IDF Vectorization. TF-IDF stands for Term Frequency–Inverse Document Frequency. It is similar to CountVectorizer, but instead of only counting how many times a word appears, it gives importance or weight to words based on how meaningful they are in the dataset.
- Words that appear frequently in one document but not in many other documents receive a higher weight. Common words that appear in many documents receive a lower weight because they are less useful for distinguishing one document from another.
- The TfidfVectorizer class is imported from Scikit-learn and then an object named tfidf_vectorizer is created. The method fit_transform(df["text"]) learns the vocabulary from the text column and converts each sentence into TF-IDF numerical values. Finally, get_feature_names_out() displays the unique words learned from the dataset.

Expected Output

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vectorizer = TfidfVectorizer()

X_tfidf = tfidf_vectorizer.fit_transform(df["text"])

print(tfidf_vectorizer.get_feature_names_out())

['am' 'bad' 'excellent' 'experience' 'happy' 'hate' 'is' 'item' 'love'
 'product' 'service' 'terrible' 'the' 'this' 'very' 'was' 'with']
```

The output shows the vocabulary learned from the dataset. The words are the same as CountVectorizer because both methods use the same input text. However, the difference is in the numerical values generated later. CountVectorizer gives simple word counts, while TF-IDF gives weighted importance scores.

Display TF-IDF Matrix

```
tfidf_df = pd.DataFrame(
    X_tfidf.toarray(),
    columns=tfidf_vectorizer.get_feature_names_out()
)

print(tfidf_df.round(3))
```

This cell converts the TF-IDF matrix into a Pandas DataFrame so that the numerical values are easier to read and understand. The method `X_tfidf.toarray()` converts the sparse TF-IDF matrix into a regular array. The vocabulary words are used as column names through `tfidf_vectorizer.get_feature_names_out()`.

The `round(3)` function rounds the decimal values to three digits, making the output cleaner and easier to interpret.

Unlike CountVectorizer, the TF-IDF matrix does not contain only 0 and 1. Instead, it contains decimal values. These values represent the importance of each word in each sentence. A value of 0 means the word is not present in that sentence. A higher value means the word is more important in that sentence.

	am	bad	excellent	experience	happy	hate	is	item	love \
0	0.00	0.000	0.000	0.000	0.00	0.000	0.000	0.000	0.703
1	0.00	0.609	0.000	0.000	0.00	0.000	0.499	0.000	0.000
2	0.42	0.000	0.000	0.000	0.42	0.000	0.000	0.000	0.000
3	0.00	0.000	0.000	0.522	0.00	0.000	0.000	0.000	0.000
4	0.00	0.000	0.703	0.000	0.00	0.000	0.576	0.000	0.000
5	0.00	0.000	0.000	0.000	0.00	0.652	0.000	0.652	0.000

	product	service	terrible	the	this	very	was	with
0	0.576	0.00	0.000	0.000	0.417	0.00	0.000	0.00
1	0.499	0.00	0.000	0.000	0.361	0.00	0.000	0.00
2	0.000	0.42	0.000	0.344	0.000	0.42	0.000	0.42
3	0.000	0.00	0.522	0.428	0.000	0.00	0.522	0.00
4	0.000	0.00	0.000	0.000	0.417	0.00	0.000	0.00
5	0.000	0.00	0.000	0.000	0.387	0.00	0.000	0.00

The output shows TF-IDF scores for each word in each sentence. For example, words like bad, excellent, happy, hate, and terrible may receive stronger scores because they help identify sentiment more clearly. This makes TF-IDF useful in real-world NLP tasks such as search engines, document classification, and text mining.

Import Porter Stemmer

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()
```

This cell imports and creates a Porter Stemmer object. Stemming is a text normalization technique used to reduce words to their base or root form. For example, words like playing, played, and plays can be reduced to play. This helps reduce the vocabulary size and allows the model to treat similar words as the same term.

The PorterStemmer is one of the most commonly used stemming algorithms in NLTK. It works by applying a set of rules to remove suffixes from words. However, stemming does not always produce a proper dictionary word. Its main goal is speed and simplification, not grammatical accuracy.

Apply Stemming

```
words = [
    "playing",
    "played",
    "plays",
    "player",
    "running",
    "runs",
    "studying",
    "studies"
]

for word in words:
    print(word, "---->", stemmer.stem(word))
```

In this cell, stemming is applied to a list of words. A list named words is created containing different forms of words such as playing, played, runs, studying, and studies. The for loop goes through each word one by one and applies the stemmer.stem() function to reduce the word to its root form.

This helps demonstrate how stemming reduces related words to a common form. For example, playing, played, and plays are all reduced to play. However, some results may not be proper English words. For example, studying and studies may become studi instead of study. This shows the limitation of stemming.

```
playing ----> play
played ----> play
plays ----> play
player ----> player
running ----> run
runs ----> run
studying ----> studi
studies ----> studi
```

The output shows that stemming is useful for reducing words, but it may sometimes produce incomplete or non-dictionary words. This is why lemmatization is often preferred when accuracy is more important.

Download WordNet Resources

```
nltk.download('wordnet')
nltk.download('omw-1.4')
```

This cell downloads the resources required for lemmatization. Lemmatization depends on vocabulary and dictionary-based analysis, so NLTK uses WordNet, a large lexical database of English words. The wordnet package helps identify the correct dictionary form of a word. The omw-1.4 package provides multilingual WordNet support and is often downloaded along with WordNet.

Without these resources, the WordNet lemmatizer may not work properly. This step is necessary before performing lemmatization.

Create WordNet Lemmatizer

```
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
```

This cell imports and creates a WordNet Lemmatizer object. Lemmatization is similar to stemming, but it is more accurate because it reduces words to their proper dictionary form. For example, studies becomes study, and cars becomes car.

The WordNetLemmatizer uses WordNet to understand the correct base form of a word. Unlike stemming, lemmatization tries to return meaningful English words. This makes it more suitable for professional NLP applications where accuracy and readability are important.

Compare Stemming and Lemmatization

```
words = [
    "playing",
    "played",
    "plays",
    "studying",
    "studies",
    "running",
    "cars"
]

for word in words:
    print(
        word,
        "---->",
        "Stem:", stemmer.stem(word),
        "| Lemma:", lemmatizer.lemmatize(word)
    )
```

This cell compares stemming and lemmatization side by side. The same list of words is passed through both the stemmer and the lemmatizer. This helps clearly show the difference between the two techniques.

Stemming applies rule-based cutting, so it may produce words like *studi*. Lemmatization uses a dictionary-based approach, so it produces more meaningful results such as *study* and *car*. However, by default, WordNetLemmatizer treats words as nouns. Because of this, some verbs such as *playing*, *played*, and *running* may not change correctly unless we explicitly specify their part of speech.

```
playing ----> Stem: play | Lemma: playing
played ----> Stem: play | Lemma: played
plays ----> Stem: play | Lemma: play
studying ----> Stem: studi | Lemma: studying
studies ----> Stem: studi | Lemma: study
running ----> Stem: run | Lemma: running
cars ----> Stem: car | Lemma: car
```

The output shows that lemmatization gives better dictionary-based results for some words, but it needs part-of-speech information to correctly process verbs.

Lemmatisation with POS Tag

```
for word in words:
    print(
        word,
        "---->",
        lemmatizer.lemmatize(word, pos="v")
    )
```

In this cell, lemmatization is performed again, but this time the part of speech is specified using `pos="v"`. Here, "v" stands for verb. By default, the lemmatizer assumes that words are nouns. This is why earlier words like *playing*, *played*, and *running* were not converted correctly. When we tell the lemmatizer to treat the words as verbs, it can produce more accurate results.

This step is important because the meaning and root form of a word can depend on its grammatical role. For example, *running* as a verb should become *run*. Providing the correct POS tag improves lemmatization accuracy.

```

for word in words:
    print(
        word,
        "---->",
        lemmatizer.lemmatize(word, pos="v")
    )

```

```

playing ----> play
played ----> play
plays ----> play
studying ----> study
studies ----> study
running ----> run
cars ----> cars

```

The output shows that verbs are now properly converted to their base forms. However, cars remains cars because we asked the lemmatizer to treat all words as verbs, and cars is normally a noun.

Download POS Tagger

```

nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')

```

This cell downloads the NLTK resources required for Part-of-Speech Tagging, also called POS tagging. POS tagging is the process of identifying the grammatical role of each word in a sentence. For example, it can identify whether a word is a noun, verb, adjective, adverb, or determiner.

The averaged perceptron tagger is a pre-trained model used by NLTK to assign POS tags to words. The second resource, averaged_perceptron_tagger_eng, may be required in newer versions of NLTK for English POS tagging.

Apply POS Tagging

```

sentence = "The dog is running fast"
tokens = word_tokenize(sentence)
pos_tags = nltk.pos_tag(tokens)
print(pos_tags)

```

- This cell performs Part-of-Speech tagging on a sample sentence. First, a sentence is stored in the variable sentence. Then, word_tokenize(sentence) splits the sentence into individual tokens. After tokenization, nltk.pos_tag(tokens) assigns a grammatical tag to each token.
- For example, the word dog is identified as a noun, is is identified as a verb, running is identified as a verb in continuous form, and fast is identified as an adverb. This helps NLP systems understand not only the words in a sentence but also their grammatical roles.
- POS tagging is useful in many NLP tasks such as information extraction, text understanding, grammar checking, chatbots, and named entity recognition.

```

sentence = "The dog is running fast"

tokens = word_tokenize(sentence)

pos_tags = nltk.pos_tag(tokens)

print(pos_tags)

```

```

[('The', 'DT'), ('dog', 'NN'), ('is', 'VBZ'), ('running', 'VBG'), ('fast', 'RB')]

```

The output is a list of tuples. Each tuple contains a word and its POS tag.

The → DT

dog → NN

is → VBZ

running → VBG

fast → RB

Here, DT means determiner, NN means noun, VBZ means verb in present tense, VBG means verb in

continuous form, and RB means adverb. This output shows that the model can identify the grammatical role of each word in the sentence.

END OF THE DOCUMENT